

EIDA Whitepaper v1.4

EIDA

Ephemeral Identity Distribution Architecture

Whitepaper v1.4

Version 1.4 — May 2026

1. Abstract

Classical Public Key Infrastructure (PKI) provides static identity guarantees — it proves that a public key belongs to an entity, but cannot guarantee that a given transaction originates from that entity at that moment. This gap enables replay attacks, key reuse vulnerabilities, and long-term compromise cascades.

EIDA (Ephemeral Identity Distribution Architecture) addresses this gap by introducing identity-level forward secrecy. The core primitive, ePriPub, is a single-use ephemeral key pair derived per transaction from the user's permanent identity, which never leaves a hardware-backed isolation zone. Once used, the ePriPub components are destroyed and the CA issues a fresh single-use certificate — no persistent state is required.

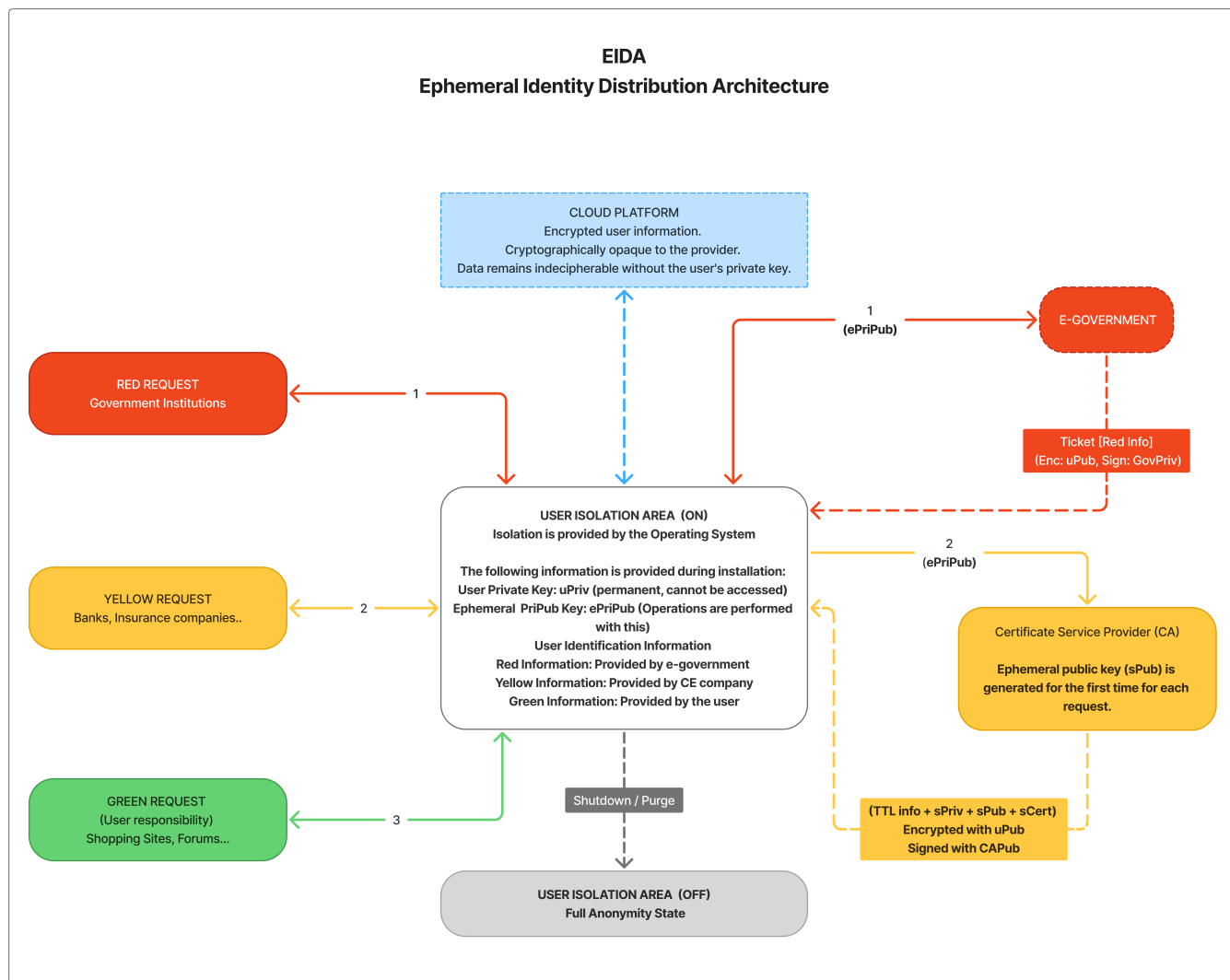
EIDA shifts the trust model from centralized authority verification to cryptographic transaction uniqueness, distributing security to the user without requiring trusted intermediaries.

2. Problem

Today, companies responsible for securing user identities are under constant attack. A single breach of a centralized security system can expose the personal data of millions of users. This is an inevitable consequence of the current architecture: security is concentrated, and concentration creates high-value targets.

EIDA proposes distributing this responsibility to user devices. In a decentralized model, an attacker who compromises a single node gains access to only one person's data — not millions. Large technology companies are no longer the primary target. The attack surface shrinks from a single critical system to millions of individual devices.

Classical PKI reinforces this centralization problem. It guarantees that a public key belongs to an entity, but cannot verify that a specific transaction originates from that entity at that moment — enabling replay attacks, key reuse, and long-term compromise cascades.



3. Architecture

EIDA consists of three layers:

1. User Isolation Zone

The user's permanent private key ($uPriv$) resides exclusively within a hardware-backed isolation zone (TPM 2.0 / TEE). It never leaves this environment under any circumstance. All cryptographic operations involving $uPriv$ are performed inside the zone.

2. $ePriPub$ Derivation Layer

For each transaction, a single-use ephemeral key pair is derived inside the isolation zone:

```
uPriv + nonce + timestamp → HKDF → seed → Ed25519 → (ePriv, ePub)
```

The nonce is randomly generated per transaction and combined with a timestamp, guaranteeing that no two derivations produce the same key pair. Transaction uniqueness is ensured by the ($ePub$, nonce, timestamp) combination — no external token coordinator is required.

3. Stateless CA Layer

The CA operates as a reactive signer — it does not scan, store, or track previous requests. Each incoming request is treated as a self-contained, atomic unit:

```
Receive: (ePub + nonce + timestamp + signature)
Verify:  signature valid? → yes
         timestamp within TTL? → yes
Generate: new (sPriv, sPub) key pair
Issue:    sCert = {sPub + expiry + metadata}, signed with CA key
Send:     (sCert + signed document) to user isolation zone
Forget:   sPriv destroyed, no state retained
```

The CA does not maintain a burned list or any persistent record of processed requests. Replay protection is provided by the short TTL window on the timestamp and the inherent single-use nature of sCert. This stateless design enables unlimited horizontal scaling — each CA node operates independently without database synchronization.

3.5 Scalability Model: Stateless Verification and the End of Revocation Lists

The Legacy Bottleneck

Traditional PKI and identity systems suffer from what can be termed the *State Exhaustion Problem*. To ensure that a token or key has not been reused or compromised, a Certificate Authority must maintain large, continuously growing Certificate Revocation Lists (CRLs) or respond to real-time status queries (OCSP). This creates a single point of latency and a database overhead that scales poorly as user populations grow into the hundreds of millions.

The EIDA Model: Zero-State Scalability

EIDA eliminates the need for a burned list or any centralized database lookup during verification. Rather than scaling the CA's state management capacity, EIDA moves the burden of security from the CA to time and hardware:

1. Hardware-Enforced Ephemerality

The ePriPub pair is generated inside a hardware-backed isolation zone (TPM/TEE) and is strictly bound to a single transaction context. The key pair ceases to exist on the client side immediately after signing — not by policy, but by design. There is nothing to revoke because there is nothing to persist.

2. Temporal Auto-Invalidation

Every EIDA token is cryptographically bound to a high-precision timestamp. The CA does not revoke keys; it simply rejects any request where $\text{Current_Time} - \text{Timestamp} > \text{TTL}$. Invalidation is a mathematical consequence of time, not a database operation.

3. O(1) Verification Complexity

The CA performs a purely mathematical verification: it checks the signature and the clock. No database is queried, no list is consulted. Verification complexity is $O(1)$ — the operation takes the same time whether the system serves one hundred or one hundred billion users.

Architectural Note: In EIDA, security is not a record maintained in a database — it is a property of the mathematics and the hardware. The CA functions as a stateless gatekeeper, making the system structurally immune to the scalability bottlenecks that constrain modern PKI deployments.

4. ePriPub — Ephemeral Primary Public Key Primitive

ePriPub is the core primitive of EIDA. It is not a protocol — it is a single-use cryptographic identity unit that carries both signing and verification capability for exactly one transaction.

4.1 Definition

ePriPub is an ephemeral key pair (ePriv, ePub) where:

- **ePriv** — ephemeral private component, used once to sign the consent token, destroyed immediately after
- **ePub** — ephemeral public component, sent to CA for signature verification, discarded after use

Neither component is stored anywhere after the transaction completes.

4.2 Derivation

```
Inputs:
  uPriv      - permanent user private key (never leaves isolation zone)
  nonce      - 32-byte cryptographically random value (generated per
transaction)
  timestamp  - Unix timestamp in milliseconds (generated per transaction)

Process:
  seed = HKDF(uPriv + nonce + timestamp)
  (ePriv, ePub) = Ed25519_derive(seed)

Output:
  (ePriv, ePub) - valid for this transaction only
```

4.3 Lifecycle

1. DERIVE — (ePriv, ePub) generated inside isolation zone
2. SIGN — consent token (ePub + nonce + timestamp) signed with ePriv
3. TRANSMIT — (ePub + nonce + timestamp + signature) sent to CA
4. VERIFY — CA verifies signature and timestamp TTL
5. DESTROY — ePriv destroyed inside isolation zone
6. GENERATE — CA generates new session key pair (sPriv, sPub)
7. SIGN-DOC — CA signs document with sPriv, issues sCert
8. RESPOND — (sCert + signed document) sent to user isolation zone

- ```

9. FORWARD — isolation zone forwards to requesting institution
10. CLEAN — sPriv destroyed by CA

```

## 4.4 Security Properties

- **Single-use** — sCert expires after TTL, replay attacks blocked by timestamp
- **Identity-bound** — derivation chain links ePriPub to uPriv without exposing it
- **Forward secrecy** — compromise of any ePriPub reveals nothing about uPriv or other transactions
- **Stateless CA** — CA retains no state between requests, enabling unlimited horizontal scaling

## 4.5 ePriPub as User Consent Proof

In classical e-signature models, the user directly signs the document with their private key. EIDA separates this into two distinct responsibilities:

```

User → proves consent (ePriPub)
CA → signs document (sPriv)

```

This separation is intentional. The CA generates a session key pair (sPriv, sPub) once per transaction and handles the actual document signature. However, the CA will only proceed with signing if a valid, unused ePriPub is present in the request.

Therefore ePriPub serves as a **cryptographic consent token**:

```

Valid ePriPub received → user consented → CA signs and issues sCert
No ePriPub → no consent proof → CA rejects
Expired timestamp → TTL window passed → CA rejects

```

### Non-repudiation guarantee:

The user cannot later deny having initiated the transaction because:

- ePriPub is derived from uPriv — only the user could have produced it
- ePriPub is single-use — it cannot have been captured and replayed
- (ePub + nonce + timestamp) combination is unique — it cannot be reused in another context

The CA cannot forge user consent because:

- CA cannot produce a valid ePriPub without access to uPriv
- uPriv never leaves the user's isolation zone

## 5. Security Analysis

### 5.1 Threat Model

EIDA considers the following adversaries:

- **Network attacker** — intercepts transmitted data
- **CA attacker** — compromises the certificate authority
- **Device attacker** — gains access to the user's device
- **Replay attacker** — attempts to reuse captured transactions

## 5.2 Attack Scenarios

### Network interception

```
Attacker captures: (ePub + nonce + timestamp + signature)
timestamp TTL → already expired (short window)
Replay attempt → CA rejects: timestamp out of TTL
uPriv → never transmitted → not exposed
Result: attack fails
```

### CA compromise

```
Attacker gains access to CA
CA stores: no persistent user data, no burned list
sPriv keys → destroyed after each transaction → not recoverable
uPriv → never reaches CA → not exposed
Result: attacker finds nothing of value
```

### Device compromise

```
Attacker gains access to user device
uPriv → inside TPM/TEE → cannot be extracted
Past ePriPub pairs → destroyed → not recoverable
Result: attacker is limited to future transactions
 only if physical device is held
```

### Replay attack

```
Attacker captures valid (ePub + nonce + timestamp + signature)
CA checks timestamp → TTL window expired → rejected
Even within TTL: sCert already delivered and used by institution
Institution validates sCert expiry → rejected
Result: attack fails
```

## 5.3 What EIDA Does Not Claim

- EIDA does not protect against physical device seizure where TPM is bypassed at hardware level
- EIDA does not provide anonymity — uPriv is permanently linked to user identity
- EIDA does not address CA availability — a downed CA blocks transactions

---

## 6. Comparison with Existing Approaches

### 6.1 Classical PKI

| Property            | Classical PKI | EIDA            |
|---------------------|---------------|-----------------|
| Identity guarantee  | Static        | Per-transaction |
| Key reuse           | Yes           | Never           |
| uPriv exposure risk | High          | None            |
| Replay protection   | External      | Native          |
| CA breach impact    | Critical      | Minimal         |
| Forward secrecy     | Session-level | Identity-level  |
| Centralization      | High          | None            |

### 6.2 TLS Ephemeral (TLS 1.3)

TLS 1.3 introduces ephemeral session keys for forward secrecy. However:

- Ephemeral keys apply to the **session layer**, not the identity layer
- Server certificate remains static and reused across all sessions
- Compromise of server certificate affects all past and future sessions

EIDA applies the ephemeral principle to the **identity layer itself** — the layer TLS never touches.

### 6.3 One-Time Signature Schemes (Lamport, WOTS)

One-time signature schemes share the single-use property with ePriPub. However:

- They were designed for **post-quantum resistance**, not identity isolation
- They do not address the centralization problem
- They have no isolation zone model or stateless CA mechanism

ePriPub is architecturally distinct — its purpose is identity-level forward secrecy, not quantum resistance.

### 6.4 Decentralized Identity (DID, W3C)

W3C DID standard moves identity off centralized registries. However:

- DID does not address per-transaction key isolation
- DID keys are reused across transactions
- No single-use consent token primitive exists in the DID model

EIDA is complementary to DID — ePriPub could serve as the signing primitive within a DID framework.

---

## 7. Conclusion

The fundamental vulnerability of current internet security architecture is concentration. When identity verification depends on centralized systems, those systems become high-value targets. A single successful attack exposes millions. This is not an implementation failure — it is an architectural one.

EIDA addresses this at the architectural level by introducing three properties that classical PKI does not provide simultaneously:

- **Identity-level forward secrecy** — each transaction uses a unique, derived key pair that is destroyed after use
- **Zero uPriv exposure** — the permanent private key never leaves the hardware-backed isolation zone
- **Distributed attack surface** — there is no central repository of user identity material worth attacking

The core primitive, ePriPub, achieves these properties by combining well-established cryptographic standards — HKDF (RFC 5869) and Ed25519 — in a novel architectural pattern. No new cryptographic assumptions are required.

EIDA does not replace existing infrastructure. It extends it. CA systems, PKI chains, and existing identity frameworks remain valid — EIDA adds a per-transaction isolation layer on top of them.

The shift EIDA proposes is conceptual as much as technical: security should not be something done to users by centralized authorities. It should be something that lives with the user, on the user's device, under the user's control.

---

## 8. References

1. RFC 5869 - HMAC-based Extract-and-Expand Key Derivation Function (HKDF)
2. RFC 8032 - Ed25519 and Ed448 Digital Signature Algorithms
3. Bernstein, D.J. et al. (2012). High-speed high-security signatures. *Journal of Cryptographic Engineering*
4. W3C (2022). Decentralized Identifiers (DIDs) v1.0
5. Rescorla, E. (2018). The Transport Layer Security (TLS) Protocol Version 1.3, RFC 8446

---

*EIDA Whitepaper v1.4*

<https://github.com/guvenacar/EIDA>